

```

package org.hightops;

import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;

public class WordCount extends Configured implements Tool {

    public int run(String[] args) throws Exception {
        // ...
    }

    public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {
        // ...
    }
}

```

Figure 4: WordCount

```

package org.hightops;

import java.io.IOException;

public class SumReducer extends Reducer<Text, Iterable<Text>, Text, Text> {

    public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {
        // ...
    }
}

```

Figure 5: SumReducer

```

package org.hightops;

import java.io.IOException;

public class WordMapper extends Mapper<String, Text, Text, Text> {

    public void map(String key, Text value, Context context) throws IOException, InterruptedException {
        // ...
    }
}

```

Figure 6: WordMapper

```

<value>org.apache.hadoop.mapreduce.ShuffleHandler</value>
</property>

```

4. In `mapred-site.xml`, copy the `mapred-site.xml.template` and rename it as `mapred-site.xml` before adding the following between configuration tabs:

```

<property> <name>mapreduce.framework.name</name>
<value>yarn</value></property>

```

5. In `hdfs-site.xml` add the following between configuration tabs:

```

<property> <name>dfs.replication</name> <value>1</value></property>
<property> <name>dfs.namenode.name.dir</name> <value>file:/home/hduser/mydata/hdfs/namenode</value></property>
<property> <name>dfs.datanode.data.dir</name> <value>file:/home/hduser/mydata/hdfs/datanode</value></property>

```

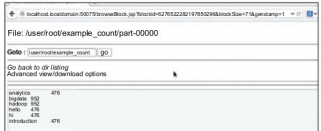


Figure 7: OutputBrowser

```

</property>

```

6. Finally, update your `.bashrc` file.

Append the following lines in the end, save and exit.

```

#Hadoop variables
export JAVA_HOME=/usr/lib/jvm/jdk/<your java version>
export HADOOP_INSTALL=/usr/local/hadoop
export PATH=$PATH:HADOOP_INSTALL/bin
export PATH=$PATH:HADOOP_INSTALL/sbin
export HADOOP_MAPRED_HOME=$HADOOP_INSTALL
export HADOOP_COMMON_HOME=$HADOOP_INSTALL
export HADOOP_HDFS_HOME=$HADOOP_INSTALL
export YARN_HOME=$HADOOP_INSTALL

```

After all this, let's make the directory for the name node and data node, for which you need to type the command `hdfs namenode -format` in the terminal. To start Hadoop and Yarn services, type `start-dfs.sh` and `start-yarn.sh`.

Now the entire configuration is done and Hadoop is up and running. We will write a Java file in Eclipse to find the number of words in a file and execute it through Hadoop. The three Java files are (Figures 4, 5, 6):

- WordCount.java
- SumReducer.java
- WordMapper.java

Now create the JAR for this project and move this to the Ubuntu side. In the terminal, execute the `jar` file with the following command `hadoop jar new.jar WordCount example.txt Word_Count_sum example.txt` is the input file (its number of words need to be counted). The final output will be shown in the `Word_count_sum` folder as shown in Figure 7.

Finally, the word count example shows the number of times a word is repeated in the file. This is but a small example to demonstrate what is possible using Hadoop on Big Data. **END** 🐧

By: Ashish Sinha

The author is a software engineer based in Bengaluru. A software enthusiast by heart, he is passionate about using open source technology and sharing it with the world. He can be reached at ashi.sinha.87@gmail.com